

■ Fire & Smoke Detection

End-to-End Project Plan — YOLOv11 (n & s), Streamlit/Gradio Demo, Hugging Face Spaces

Project root	~/Desktop/project/Fire AI
Base dataset	D-Fire (17,221 train / 4,306 test images, YOLO format, 2 classes)
Models	YOLOv11n (nano) and YOLOv11s (small)
Compute	Local — Apple M3, 8 GB unified memory, MPS backend
Deliverables	GitHub repo, 1-min demo video, README with mAP@0.5 & mAP@0.5:0.95, hosted HF Spaces de

1. Project Overview

Goal: train an object detector that finds **fire** and **smoke** in images/video, compare a nano vs. small YOLOv11 model on accuracy vs. speed, and ship it as a working public demo with a documented, reproducible repo.

Final deliverables checklist

- GitHub repository with clean structure, code, and documentation
- Trained weights for YOLOv11n and YOLOv11s (best.pt each), tracked via Releases or Git LFS (not committed raw into git history)
- README.md with a results table: mAP@0.5 and mAP@0.5:0.95 for both models on the held-out test set
- A 1-minute video showing the detector running on test footage (screen recording of the demo app or a script running on a video file)
- A hosted demo on Hugging Face Spaces (free CPU tier) that anyone can open in a browser
- ~100-200 personally collected & annotated images (CVAT or Roboflow) used as a secondary real-world validation/demo set

Key decisions already made with you

- **Data:** use D-Fire as the primary training set (it is already YOLO-labeled, ~21.5k images total). You will additionally shoot and annotate ~100-200 of your own images/frames in CVAT or Roboflow so you get real annotation practice and have a genuine real-world test clip for the demo video.
- **Compute:** training happens locally on your MacBook Air M3 (8 GB unified memory) using PyTorch's MPS backend. This is slower than an NVIDIA GPU and memory-constrained, so the plan below uses small batch sizes, modest image size, and realistic epoch budgets. Google Colab is kept as a fallback if local training is too slow.

2. Dataset Audit — What You Already Have

I inspected data/raw/D-Fire directly. Findings:

Split	Images	Labels	Format
train	17,221	17,221 (.txt)	labels_clean/ used (deduplicated/fixed labels)
test	4,306	4,306 (.txt)	labels_clean/ used

- Labels are standard YOLO format: `class x_center y_center width height` (normalized 0-1).
- Two classes present: 0 and 1. In the public D-Fire dataset these correspond to **smoke** and **fire** — we will confirm the exact mapping empirically in Phase 1 by rendering sample boxes, since this is a common source of silent bugs.
- 7,833 of the 17,221 train label files are empty — these are background/negative images (no fire or smoke). Keeping a healthy fraction of negatives is good for reducing false positives, so we keep them.

- All images are .jpg. Dataset folder is ~3.0 GB on disk.
- There is no built-in validation split — only train/test. We will carve a val split out of train (e.g. 90/10) for use during training, and reserve the official test/ folder purely for final, untouched evaluation (this is what generates the mAP numbers for the README).

Note: Do not train on any part of test/. It exists only to produce the final, honest mAP@0.5 and mAP@0.5:0.95 numbers you report.

3. Environment Setup

3.1 Create an isolated environment

```
cd ~/Desktop/project/Fire\ AI

# Using conda (miniconda already installed on this machine)
conda create -n fireai python=3.11 -y
conda activate fireai

pip install -U pip
pip install ultralytics==8.3.* torch torchvision \
    streamlit gradio opencv-python-headless \
    matplotlib pandas pyyaml
```

3.2 Verify Apple Silicon GPU (MPS) is usable

```
python3 -c "import torch; print(torch.backends.mps.is_available())"
# -> should print True on this M3 machine
```

Note: Ultralytics will auto-detect and use device='mps' on Apple Silicon. If you hit MPS-specific bugs, fall back to device='cpu' (slower but always works) or move training to Google Colab.

3.3 Repository structure to create

```
Fire AI/
├── data/
│   ├── raw/D-Fire/           # original dataset (already present, git-ignored)
│   ├── processed/           # train/val split + data.yaml (git-ignored, large)
│   └── custom/               # your self-collected + annotated images
├── scripts/
│   ├── verify_dataset.py     # sanity-check labels, class mapping, plot samples
│   ├── make_split.py         # carve val out of train
│   ├── train.py              # thin wrapper around ultralytics training
│   └── evaluate.py           # run val on test/, dump mAP table
├── app/
│   ├── streamlit_app.py      # or gradio_app.py
│   └── requirements.txt       # deployment-only deps for HF Spaces
├── runs/                     # ultralytics training outputs (git-ignored)
├── models/                   # exported best.pt for each model (small enough to commit, or use Releases)
├── docs/
│   └── Fire_Smoke_Detection_Project_Plan.pdf # this document
├── notebooks/                # optional exploration notebooks
├── README.md
├── requirements.txt
└── .gitignore
```

4. Phase-by-Phase Plan

Phase 0 — Repo & environment (30-45 min)

- 1 git init the project, add .gitignore (exclude data/raw, data/processed, runs/, venv, __pycache__, *.pt except final chosen weights).
- 2 Create the folder skeleton above.
- 3 Set up the conda env and confirm MPS works (Section 3).

Phase 1 — Dataset preparation (1-2 hrs)

- 1 Run a verification script that (a) checks every image has a matching label file, (b) draws boxes for a random sample of class 0 and class 1 to visually confirm which class is fire vs. smoke, (c) prints class balance and box-size statistics.
- 2 Carve a validation split from train/ (e.g. 90% train / 10% val, stratified so both classes and background images are represented). Do NOT touch test/.
- 3 Write data.yaml pointing at train/val/test paths with names: {0: smoke, 1: fire} (or the confirmed correct order).
- 4 Given the 8 GB RAM constraint, decide the working training set size: start with a subset (e.g. 6,000-8,000 images) for fast iteration/debugging, then run the final full-dataset training run once the pipeline is verified.

Phase 2 — Your own annotated images (2-4 hrs, spread out)

- 1 Capture ~100-200 images: phone photos of controlled/safe fire sources (candles, matches, campfire, cooking, bonfire from a safe distance) and smoke (steam, incense, smoke machine, controlled outdoor burning) plus some negative images (no fire/smoke) similar to your intended demo environment. Prioritize your own safety — never create hazardous fires for the sake of a dataset.
- 2 Alternative/supplement: extract frames from your own phone videos of fire/smoke (1 frame per ~10-15 frames to avoid near-duplicates).
- 3 Annotate in CVAT (self-hosted or cvat.ai free tier) or Roboflow (free tier): draw bounding boxes around every visible fire and smoke region, export in 'YOLO v5/v8 PyTorch TXT' format.
- 4 Drop the exported images+labels into data/custom/. This set is used two ways: (a) optionally blended a little into val for extra diversity, (b) kept aside as the 'real world' clip for your demo video and qualitative screenshots.

Phase 3 — Train YOLOv11n (baseline)

Nano model: fastest to train, good first sanity check that the whole pipeline (data → train → val → inference) works end to end before spending hours on the larger model.

```
yolo detect train \  
  model=yolo11n.pt \  
  data=data/processed/data.yaml \  
  epochs=60 \  
  imgsz=512 \  
  batch=16 \  
  device=mps \  

```

```
patience=15 \
project=runs/detect \
name=yolo11n_fire_smoke
```

- `imgsz=512` (not 640) and `batch=16` chosen to stay inside 8 GB unified memory; drop to `batch=8` if you see MPS out-of-memory errors.
- `patience=15` enables early stopping so you don't burn hours once the model plateaus.
- Expect roughly a few hours for a first full run on the full dataset on an M3 Air — run it overnight if needed. Use the debug subset from Phase 1 first to confirm no crashes before committing to a long run.

Phase 4 — Train YOLOv11s

```
yolo detect train \
  model=yolo11s.pt \
  data=data/processed/data.yaml \
  epochs=60 \
  imgsz=512 \
  batch=8 \
  device=mps \
  patience=15 \
  project=runs/detect \
  name=yolo11s_fire_smoke
```

Same recipe as the nano model but with a smaller batch size (the 's' model uses more memory per image). This will take noticeably longer than the nano run — plan for it to run overnight or across a couple of sessions.

Phase 5 — Evaluation (produces your README numbers)

```
yolo detect val \
  model=runs/detect/yolo11n_fire_smoke/weights/best.pt \
  data=data/processed/data.yaml \
  split=test imgsz=512
```

```
yolo detect val \
  model=runs/detect/yolo11s_fire_smoke/weights/best.pt \
  data=data/processed/data.yaml \
  split=test imgsz=512
```

- Ultralytics prints/saves `mAP50` and `mAP50-95` (overall and per-class) — copy these into the README results table.
- Also record inference speed (ms/image, or FPS) for both models on this machine, to justify the nano-vs-small tradeoff discussion in the README.
- Run both models qualitatively on your custom test images (Phase 2) and save a few annotated output images for the README/demo.

Model	mAP@0.5	mAP@0.5:0.95	Params	Speed (ms/img, MPS)
YOLOv11n	<i>fill in</i>	<i>fill in</i>	~2.6M	<i>fill in</i>
YOLOv11s	<i>fill in</i>	<i>fill in</i>	~9.4M	<i>fill in</i>

Phase 6 — Build the demo app

Recommendation: Gradio, because it deploys to Hugging Face Spaces with almost zero extra configuration (Spaces has first-class Gradio support). Streamlit also works on Spaces but needs a small Dockerfile/README config block.

```
# app/gradio_app.py (sketch)
import gradio as gr
from ultralytics import YOLO

model = YOLO("models/yolo11s_best.pt") # ship the better of the two

def detect(image):
    results = model.predict(image, conf=0.25)
    return results[0].plot() # numpy array with boxes drawn

demo = gr.Interface(
    fn=detect,
    inputs=gr.Image(type="numpy"),
    outputs=gr.Image(type="numpy"),
    title="Fire & Smoke Detector (YOLOv11)",
    examples=["examples/fire1.jpg", "examples/smoke1.jpg"],
)
demo.launch()
```

- Also add a video-upload tab (iterate over frames, run detection, re-encode) so you can demo on the test footage clip.
- Test locally with `python app/gradio\_app.py` before touching Hugging Face.

Phase 7 — Deploy to Hugging Face Spaces (free tier)

- 1 Create a free account at huggingface.co, then 'New Space' → SDK: Gradio → hardware: CPU basic (free).
- 2 Push your app folder (app.py, requirements.txt, model weights, example images) to the Space's git repo (Spaces are just git repos).
- 3 CPU-only inference: YOLOv11n will feel snappier than v11s on the free tier — consider defaulting the Space to the nano model, or exposing a dropdown to pick which model to run.
- 4 Confirm the Space builds successfully and the public URL works from a fresh browser/incognito window.

Phase 8 — Record the 1-minute demo video

- 1 Prepare a short test clip: a mix of D-Fire test images/video and your own custom footage (Phase 2), showing true positives and at least one tricky/near-miss case.
- 2 Screen-record (QuickTime Player → File → New Screen Recording on macOS) the Gradio app (local or the live Space) running detection on that footage.
- 3 Trim to ~60 seconds, keep it focused: a few seconds of intro, then detections running, then a quick n-vs-s comparison shot if time allows.
- 4 Export as .mp4 and either upload to the repo (docs/demo.mp4, keep it small) or host on YouTube/Drive and link it from the README.

Phase 9 — Write the README

Sections to include:

- Problem statement & motivation (1 paragraph)
- Dataset: D-Fire citation/link + your custom-annotated subset, with counts and a couple of example images
- Method: YOLOv11n vs YOLOv11s, training config (epochs, imgsz, batch, hardware)
- Results table: mAP@0.5 and mAP@0.5:0.95 for both models (from Phase 5), plus speed comparison
- Demo: link to the Hugging Face Space + embedded/linked 1-minute video
- How to run locally (setup + inference commands)
- Limitations & future work (e.g. small/rare fire, night scenes, false positives on red/orange objects)
- Credits/license (D-Fire dataset citation, Ultralytics YOLO license note)

Phase 10 — GitHub push & polish

- 1 Do NOT commit the raw 3 GB dataset. .gitignore data/raw and data/processed; document the download/prep steps instead.
- 2 Commit final best.pt weights only if small enough (nano ~5-6MB is fine; consider GitHub Releases or Git LFS for the small model if repo size becomes an issue).
- 3 Final pass: run through README as a first-time reader, confirm every command in it actually works from a clean clone.

5. Apple M3 / 8 GB — Practical Notes

- 8 GB is unified memory shared with the OS and everything else running — close other heavy apps (browser tabs, etc.) during training.
- Prefer imgsz=512 over the YOLO default 640 to reduce memory pressure; only go back to 640 for a final higher-quality run if time/memory allow.
- If you hit 'MPS backend out of memory': lower batch size first (16 → 8 → 4), then lower imgsz, then consider training on a subsample.
- MPS training can occasionally be less numerically stable or slightly slower per-image than a discrete NVIDIA GPU of similar tier; if a run stalls or errors persistently, Google Colab's free T4 GPU is the documented fallback in this plan.
- Use patience (early stopping) and save-period checkpoints so a long overnight run isn't wasted if you need to stop it.

6. Suggested Timeline (part-time pace)

Day	Focus
1	Phase 0-1: repo, env, dataset verification, train/val split, data.yaml
2	Phase 2: capture & annotate your own ~100-200 images in CVAT/Roboflow
3	Phase 3: YOLOv11n training (kick off long run, e.g. overnight)

Day	Focus
4	Phase 4: YOLOv11s training (kick off long run, e.g. overnight)
5	Phase 5: evaluation, collect mAP table, qualitative checks
6	Phase 6-7: build demo app, deploy to Hugging Face Spaces
7	Phase 8-10: record video, write README, final GitHub polish & push

Note: Compress or stretch this freely — the important dependency order is: verify data → train n → train s → evaluate → build/deploy demo → record video → write README last (it needs the real numbers and the live demo link).

7. What Happens Next in This Session

Right after this PDF, we will start executing Phase 0 and Phase 1 together in this repo: initialize git, create the folder structure, write the dataset verification script (including the fire-vs-smoke class check), carve the train/val split, and generate data.yaml — so the very next thing you can do is kick off the YOLOv11n training run.